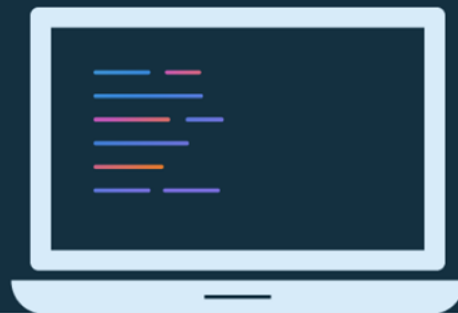




Android Development with Kotlin



Storing data

Ways to store data in an Android app

- App-specific storage
- Shared storage (files to be shared with other apps)
- Preferences
- Databases

Data Storage

- Android provides several options for you to save persistent application data.
- The solution you choose depends on your specific needs, such as whether the data should be
 - private to your application or
 - Accessible to other applications (and the user)
 - and how much space your data requires.

Data Storage Options

Option	Description
Shared Preferences	Store <i>private primitive</i> data in <i>key-value</i> pairs.
Internal Storage	Store <i>private</i> data on <i>device memory</i> .
External Storage	Store <i>public</i> data on the <i>shared external storage</i> .
SQLite Databases	Store <i>structured</i> data in a <i>private</i> database
Network Connection	Store data on the web with your own network server.

Shared Preferences

Shared Preferences

- The ***SharedPreferences*** class provides a general framework that allows you to save and retrieve persistent ***private key-value pairs*** of ***primitive data types*** to a ***file*** on the device's storage.
- Data is ***private*** to the application.
- You can use ***SharedPreferences*** to save any ***primitive data like: booleans, floats, ints, longs, and strings.***

Shared Preferences

- SharedPreferences class provides APIs for reading, writing, and managing this data
- This data will persist across user sessions (even if your application is killed)
- Save data in `onPause()` ... `onStop()`
restore in `onCreate()` ... `onStart()`

Shared Preferences VS Saved Instance State

- Persist data across user sessions, even if the app is killed and restarted, or the device is rebooted
- Data that should be remembered across sessions, such as a user's preferred settings or their game score
- Common use is to store user preferences
- Preserves state data across activity instances in the same user session
- Data that should not be remembered across sessions, such as the currently selected tab or current state of activity.
- Common use is to recreate the state after the device has been rotated

Creating Shared Preferences

- To get a **SharedPreferences** object for your application, use one of two methods:
 1. `getSharedPreferences(name: String, mode: Int): SharedPreferences`
 - Use this if you need multiple preferences files identified by name, which you specify with the first parameter.
 - Name it with the package name of your app
 - unique and easy to associate with the app

Creating Shared Preferences

- To get a **SharedPreferences** object for your application, use one of two methods:
 1. `getSharedPreferences(name: String, mode: Int): SharedPreferences`
 - `MODE` argument for `getSharedPreferences()` is for backwards compatibility, use only `MODE_PRIVATE` to be secure
 - If a preferences file by this name does not exist, it will be created when you retrieve an editor `SharedPreferences.edit()` and then commit changes `Editor.commit()`

Shared Preferences cont'd

- To get a **SharedPreferences** object for your application, use one of two methods:
 2. `getPreferences()`
 - Use this if you need only one preferences file for your Activity.
 - Because this will be the only preferences file for your Activity, you don't supply a name.

Saving Shared Preferences

- [SharedPreferences.Editor](#) Interface
- Takes care of all file operations
- put methods overwrite if the key exists
 - `apply()`
 - saves asynchronously and safely
 - does not guarantee that the changes will immediately persist on the disk
 - does not return a Boolean indicating the success of the save operation

Saving Shared Preferences cont'd

- [SharedPreferences.Editor](#) Interface
- Takes care of all file operations
- put methods overwrite if the key exists
 - `commit()`
 - saves synchronously
 - guarantees that the changes will immediately persist on the disk
 - returns a Boolean indicating whether the save operation was successful or not

Saving Shared Preferences

Writing

- Call `edit()` to get `SharedPreferences.Editor`
- Add values with methods such as:
 - `putBoolean(key: String, value: Boolean)`
 - `putString(key: String, value: String)`
- Commit the new values with `commit()`

Edit Shared Preference cont'd

Read Methods

- `getBoolean(key: String, defaultValue: boolean): Boolean`
- `getString(key: String, defaultValue: String): String`

SharedPreferences.Editor

```
class MainActivity : AppCompatActivity() {  
    private val PREF_NAME: String = "File Name"  
    private val sharedPref =  
        sharedPreferences(PREF_NAME, MODE_PRIVATE)  
    override fun onPause() {  
        super.onPause()  
  
        val editor = sharedPref.edit()  
        editor.putInt("saved_high_score_key", newHighScore)  
        editor.apply()  
    }  
}
```

SharedPreferences.Editor

```
class MainActivity : AppCompatActivity() {  
    private val PREF_NAME: String = "File Name"  
    private val sharedPref =  
        sharedPreferences(PREF_NAME, MODE_PRIVATE)  
    override fun onPause() {  
        super.onPause()  
  
        with(sharedPref.edit()) {  
            putInt("saved_high_score_key", newHighScore)  
            apply()  
        }  
    }  
}
```

Restoring Shared Preferences

- Restore in onCreate() in Activity
- Get methods to take two arguments
the *key*, and the *default value* if the key cannot be found
- Use default argument so you do not have to test
whether the preference exists in the file

Getting data in onCreate()

```
private val defaultValue = 0

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val highScore = sharedPref.getInt("saved_high_score_key",
                                      defaultValue)
}
```

Clearing

- Call `clear()` on the `SharedPreferences.Editor` and apply changes
- You can combine calls to `put` and `clear`. However, when you `apply()`, `clear()` is always done first, regardless of order!

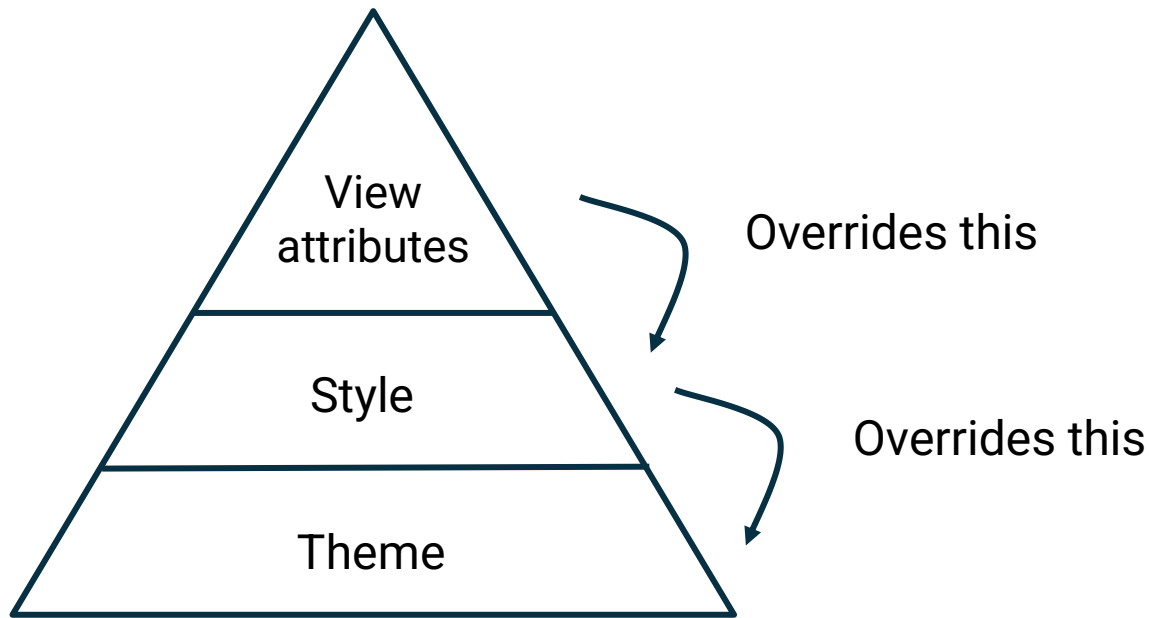
```
val preferencesEditor = sharedPref.edit()  
preferencesEditor.clear()  
preferencesEditor.apply()
```

Android styling

Android styling system

- Used to specify the visual design of your app
- Helps you maintain a consistent look across your app
- Hierarchical
(you can inherit from parent styles and override specific attributes)

Precedence of each method of styling



Themes

- Collection of named resources, useful broadly across the app
- Named resources are known as theme attributes
- Examples:
 - Use a theme to define primary & secondary colors in the app
 - Use a theme to set the default font for all text within an activity

Declare a theme

In `res/values/themes.xml`:

```
<style name="Theme.MyApp" parent="Theme.MaterialComponents.Light">
    <item name="colorPrimary">@color/orange_500</item>
    <item name="colorPrimaryVariant">@color/orange_700</item>
    <item name="colorSecondary">@color/pink_200</item>
    <item name="colorSecondaryVariant">@color/pink_700</item>
    ...
</style>
```

Apply a theme

In `AndroidManifest.xml`:

```
<manifest ... >
    <application ... >
        <activity android:theme="@style/Theme.MyApp" ... >
            </activity>
        </application>
    </manifest>
```

In layout file:

```
<ConstraintLayout ...
    android:theme="@style/Theme.MyApp">
```

Refer to theme attribute in a layout

In layout file:

```
<LinearLayout ...  
    android:background="?attr/colorSurface">
```

Use `?attr/themeAttributeName` syntax.

Examples: `?attr/colorPrimary`
`?attr/colorPrimaryVariant`

Styles

- A style is a collection of view attributes, ***specific to a type of view***
- Use a style to create a collection of reusable styling information, such as font size or colors
- Good for declaring small sets of common designs used throughout your app

Declare a style

In `res/values/styles.xml`:

```
<style name="DescriptionStyle">
    <item name="android:textColor">#00FF00</item>
    <item name="android:textSize">16sp</item>
    ...
</style>
```

Apply a style

On a view in a layout file:

```
<TextView
    style="@style/DescriptionStyle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/description_text" />
```

Refer to theme attribute in a style

In `res/values/styles.xml`:

```
<style name="DescriptionStyle">
    <item name="android:textColor">?attr/colorOnSurface</item>
    <item name="android:textSize">16sp</item>
    ...
</style>
```

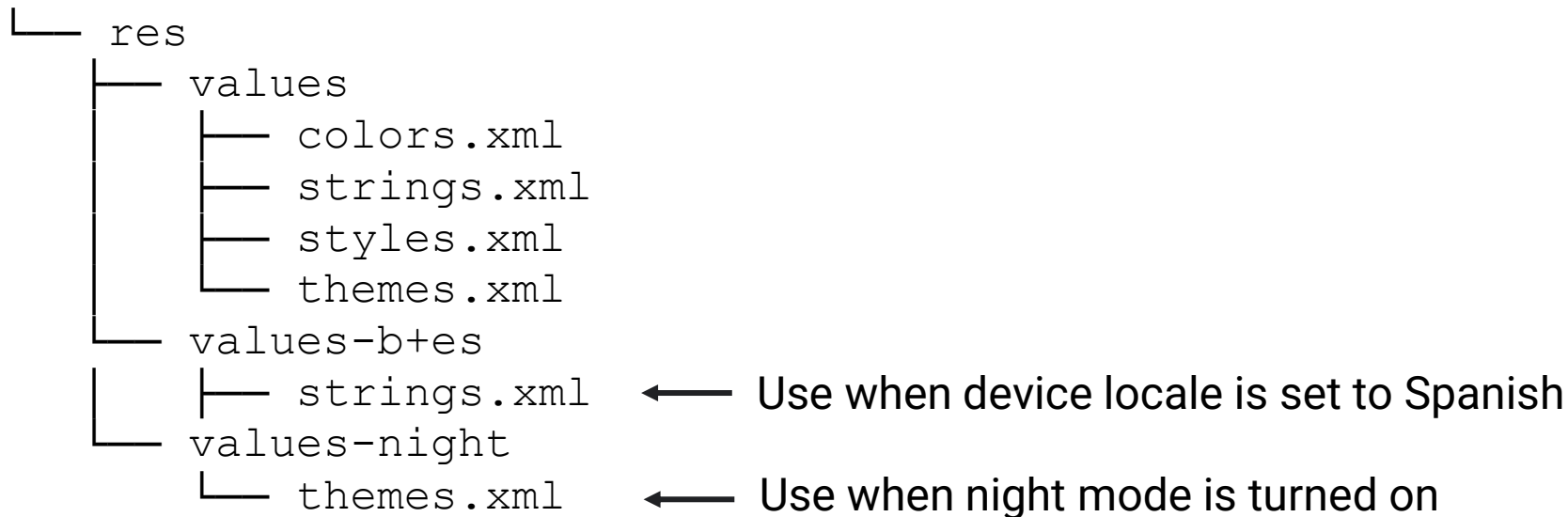
View attributes

- Use view attributes to set attributes explicitly for each view
- You can use every property that can be set via styles or themes
- Use for custom or one-off designs such as margins, paddings, or constraints

Resources directory

```
└─ res
    ├── drawable
    ├── drawable-*
    ├── layout
    ├── menu
    ├── mipmap-*
    ├── navigation
    ├── values
    │   ├── colors.xml
    │   ├── dims.xml
    │   ├── strings.xml
    │   ├── styles.xml
    │   └── themes.xml
    └── values-*
```

Provide alternative resources



Color resources

A way to name and standardize colors throughout your app

In `res/values/colors.xml`:

```
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    ...
</resources>
```

Specified as hexadecimal colors in form of #AARRGGBB

Dimension resources

A way to name and standardize dimension values in your layouts

- Declare your dimension values in `res/values/dimens.xml`:

```
<resources>
    <dimen name="top_margin">16dp</dimen>
</resources>
```

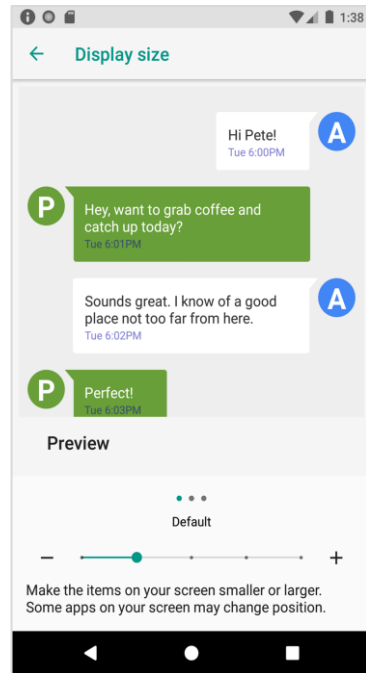
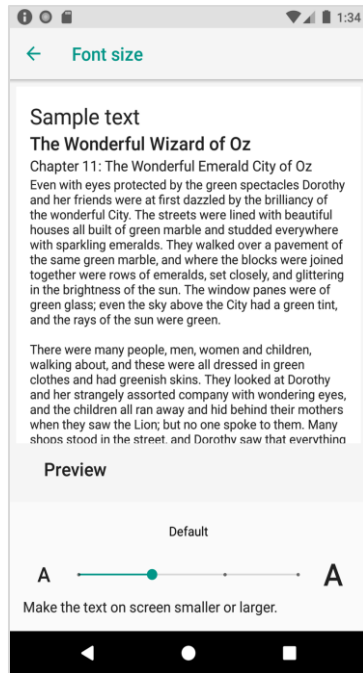
- Refer to them as `@dimen/<name>` in layouts or `R.dimen.<name>` in code:

```
<TextView ...
    android:layout_marginTop="@dimen/top_margin" />
```

Typography

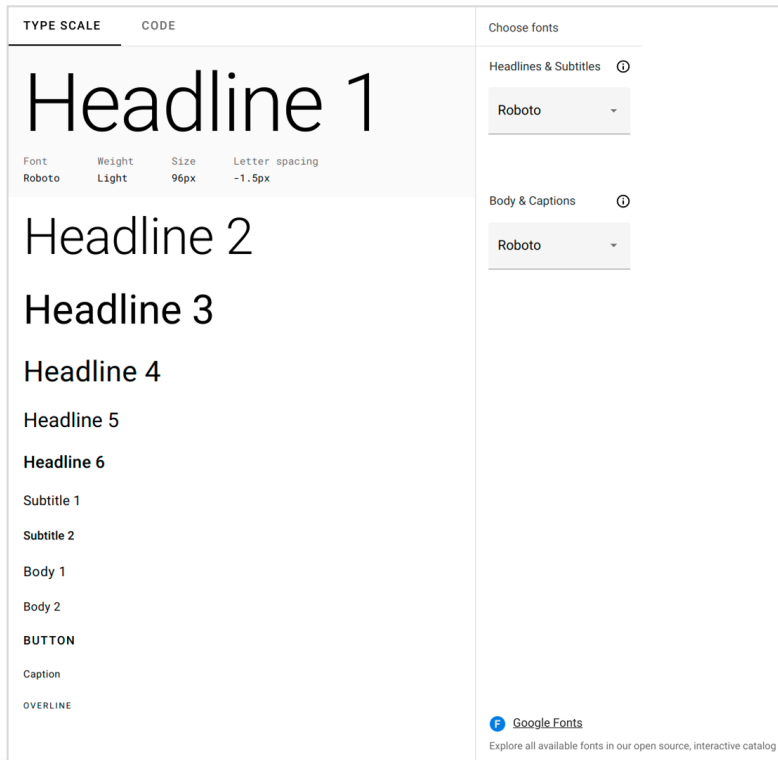
Scale-independent pixels (sp)

- The textual equivalent to density-independent pixels (dp)
- Specify text sizes in sp (takes into account user preferences)
- Users can adjust Font and Display sizes in the Settings app (after Display)



Type scale

- A set of styles designed to work together in a cohesive manner for your app and content
- Contains reusable categories of text with intended purpose for each (for example, headline, subtitle, caption)



TextAppearance

A `TextAppearance` style often alters one or more of these attributes:

- `typeface` (`android:fontFamily`)
- `weight` (`android:textStyle`)
- `text size` (`android:textSize`)
- `capitalization` (`android:textAllCaps`)
- `letter spacing` (`android:letterSpacing`)

Examples using TextAppearance

```
<TextView
```

```
    ...  
    android:textAppearance="@style/TextAppearance.MaterialComponents.Headline1"  
    android:text="@string/title" />
```

```
<TextView
```

```
    ...  
    android:textAppearance="@style/TextAppearance.MaterialComponents.Body1"  
    android:text="@string/body_text" />
```

Customize your own TextAppearance

```
<style name="TextAppearance.MyApp.Headline1"
    parent="TextAppearance.MaterialComponents.Headline1">
    ...
    <item name="android:textStyle">normal</item>
    <item name="android:textAllCaps">false</item>
    <item name="android:textSize">64sp</item>
    <item name="android:letterSpacing">0</item>
    ...
</style>
```

Use a custom TextAppearance in a theme

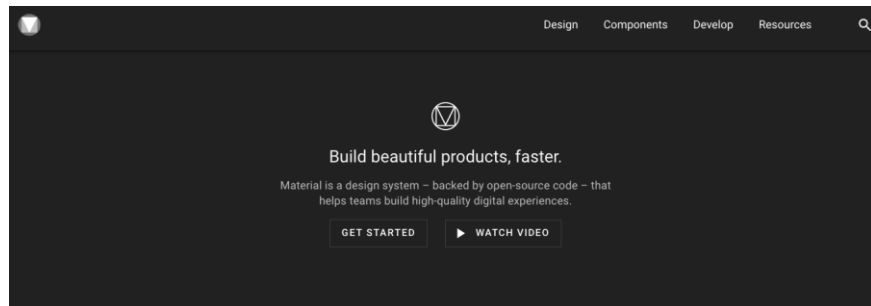
```
<style name="Theme.MyApp" parent="Theme.MaterialComponents.Light">  
    ...  
    <item name="textAppearanceHeadline1">@style/TextAppearance.MyApp.Headline1</item>  
    ...  
</style>
```

Material Design

Intro to Material

Adaptable system of guidelines, components, and tools that support best practices for UI design

[Material Design homepage](#)



Design guidance and code

Use our most popular design and development resources to jumpstart your latest project



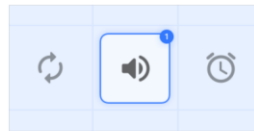
Material Design guidelines

Material Design principles, styles, and best practices



Components

Design guidance and developer documentation for interactive UI building blocks

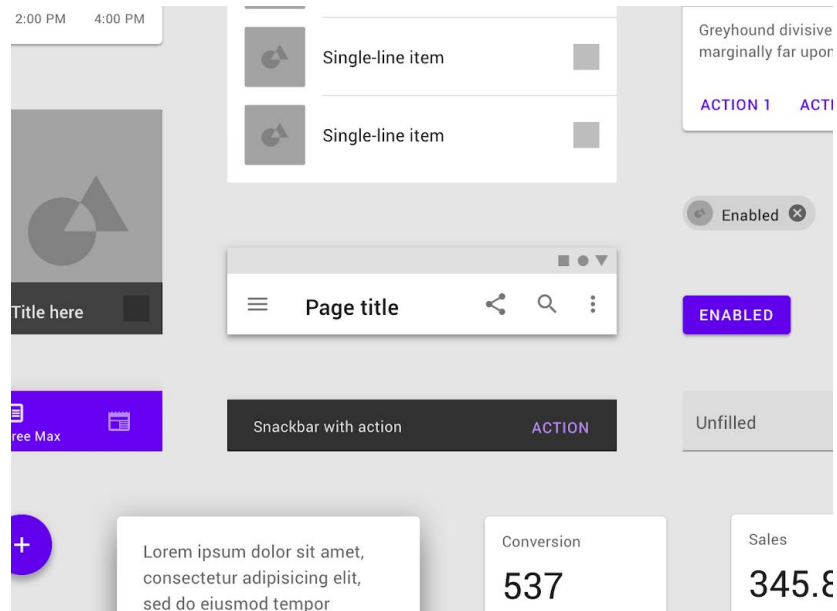


Icons

Access five sets of stylized system icons, available in a range of formats and sizes

Material Components

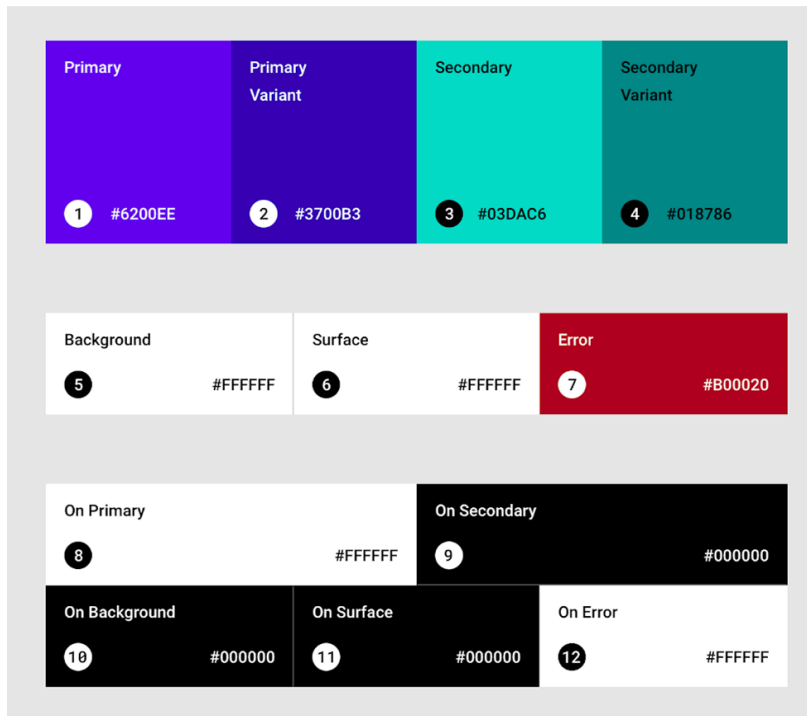
Interactive building blocks
for creating a user interface



Material color tool

The screenshot displays the Material Color Tool interface. On the left, a preview of a user interface is shown with a purple header bar containing the text "Text", a large grey rectangular area, and a bottom section with a purple circular button containing a white plus sign and a horizontal slider below it. The top of the tool features a "COLOR TOOL" header with a red and black icon, and navigation tabs for "USER INTERFACES" and "ACCESSIBILITY". A "1/6" indicator is visible between the tabs. On the right, a "MATERIAL PALETTE" is shown with a grid of color swatches organized by hue (Red, Pink, Purple, Deep Purple, Indigo, Blue, Light Blue, Cyan, Teal) and value (50, 100, 200, 300, 400, 500, 600, 700, 800, 900). A "CUSTOM" tab is also present. Below the palette, the "CURRENT SCHEME" is displayed, showing a primary color of #5d00ed and a secondary color of #ffffff. A "RESET ALL" button is located to the right of the current scheme. The current scheme also includes a "Text on P" section with a purple background and white text "T", and a "Text on S" section with a white background and purple text "T". The primary color section includes a "RESET" button and two color swatches: "P - Light #9946ff" and "P - Dark #0000b9". The secondary color section includes two color swatches: "S - Light" and "S - Dark".

Baseline Material color theme



Material Components for Android Library

```
implementation 'com.google.android.material:material:<version>'
```

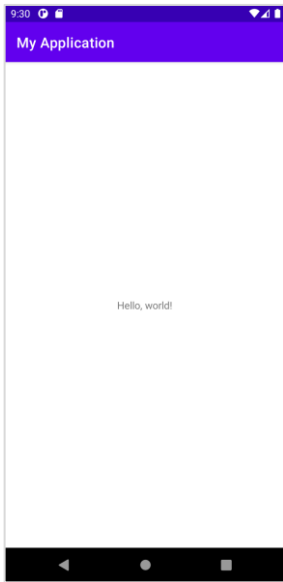
Material Themes

- `Theme.MaterialComponents`
- `Theme.MaterialComponents.NoActionBar`
- `Theme.MaterialComponents.Light`
- `Theme.MaterialComponents.Light.NoActionBar`
- `Theme.MaterialComponents.Light.DarkActionBar`
- `Theme.MaterialComponents.DayNight`
- `Theme.MaterialComponents.DayNight.NoActionBar`
- `Theme.MaterialComponents.DayNight.DarkActionBar`

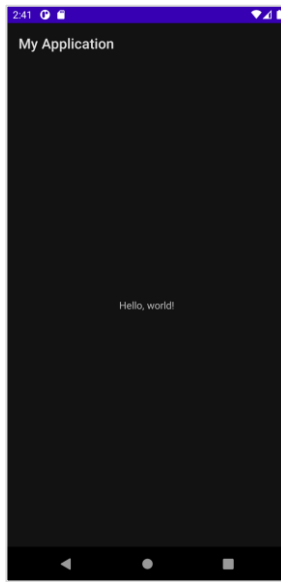
Material theme example

`Theme.MaterialComponents.DayNight.DarkActionBar`

Light mode



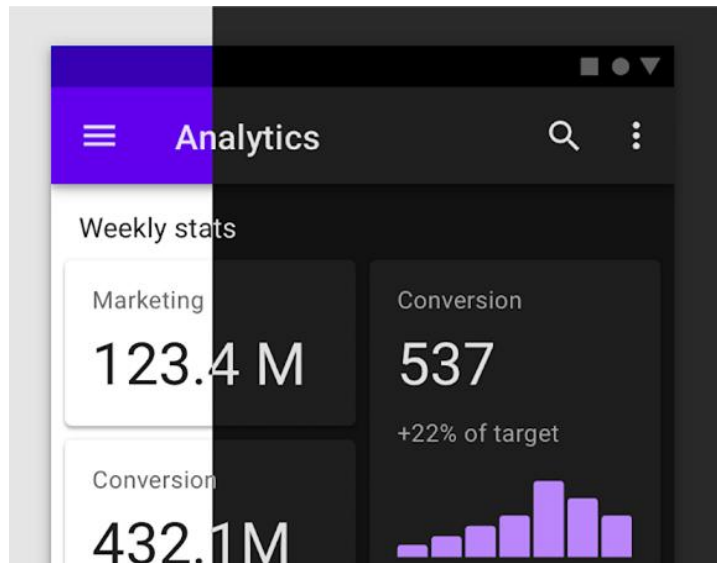
Dark mode



Dark theme

A low-light UI that displays mostly dark surfaces

- Replaces light-tinted surfaces and dark text with dark-tinted surfaces and light text
- Makes it easier for anyone to use a device in lower-light environments
- Improves visibility for users with low vision and those sensitive to bright light
- Can significantly reduce power usage (depending on the device)



Support dark theme

In `values/themes.xml`:

```
<style name="AppTheme" parent="Theme.MaterialComponents.DayNight">  
    <item name="colorPrimary">@color/orange_500</item>  
    ...
```

In `values-night/themes.xml`:

```
<style name="AppTheme" parent="Theme.MaterialComponents.DayNight">  
    <item name="colorPrimary">@color/orange_200</item>  
    ...
```



Material Components

Material Components

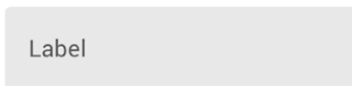
Component library provided for Android and design guidelines

- Text fields
- Buttons
- Menus
- Cards
- Chips
- App bars (top and bottom)
- Floating Action Button (FAB)
- Navigation Drawer
- Bottom navigation
- Snackbar

...and more!

Text field

- Composed of `TextInputLayout` with child view `TextInputEditText`
- Shows a floating label or a text hint before the user enters text
- Two types:



Filled text field



Outlined text field

Text field example

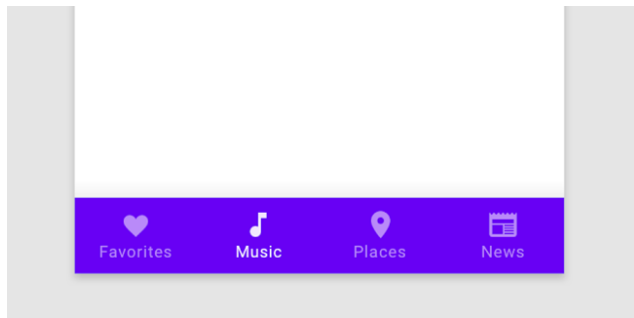
```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/textField"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="@string/label"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox">

    <com.google.android.material.textfield.TextInputEditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</com.google.android.material.textfield.TextInputLayout>
```

Bottom navigation

- Allows movement between top level destinations in your app
- Alternate design pattern to a navigation drawer
- Limited to 5 locations max



Bottom navigation example

```
<LinearLayout ...>
```

```
...
```

```
<com.google.android.material.bottomnavigation.BottomNavigationView  
    android:id="@+id/bottom_navigation"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    app:menu="@menu/bottom_navigation_menu" />
```

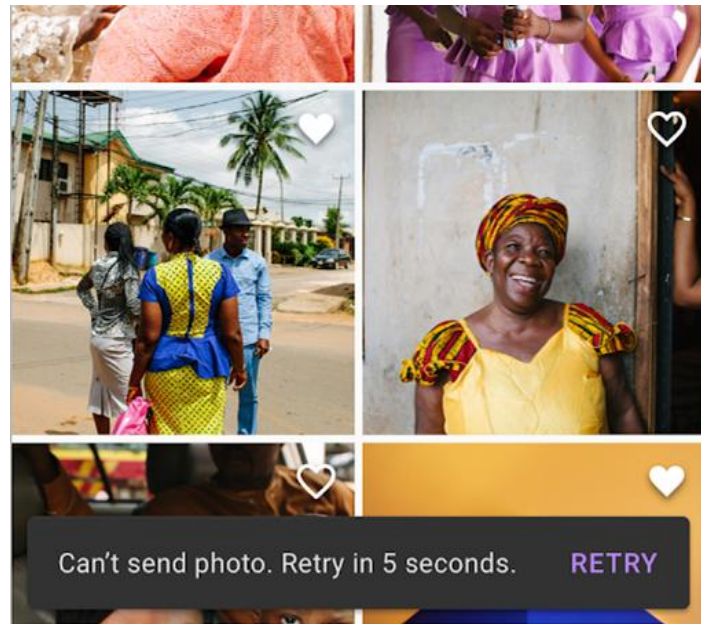
```
</LinearLayout>
```

Bottom navigation listener

```
bottomNav.setOnNavigationItemSelectedListener { item ->
    when(item.itemId) {
        R.id.item1 -> {
            // Respond to navigation item 1 click
            true
        }
        R.id.item2 -> {
            true
        }
        else -> false
    }
}
```

Snackbar

- Display short messages within the app
- Messages have a duration (SHORT, LONG, or INDEFINITE)
- May contain an optional action
- Works best in a `CoordinatorLayout`
- Shown at bottom of enclosing container



Snackbar examples

Show a simple message:

```
Snackbar.make(view, R.string.text_label, Snackbar.LENGTH_SHORT)
    .show()
```

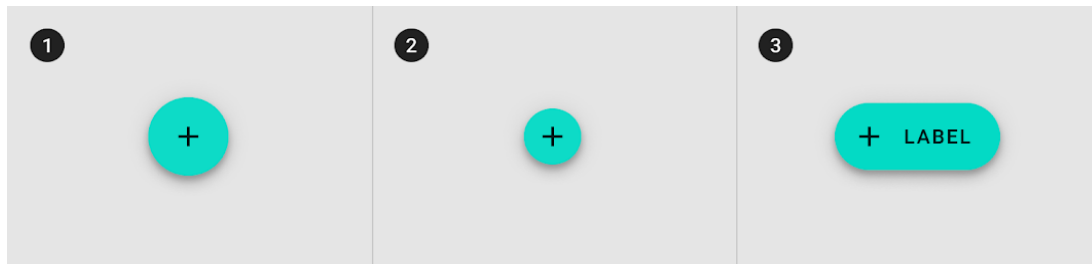
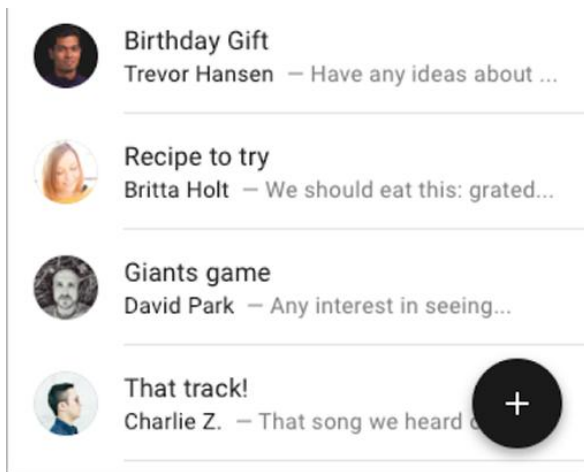
Add an action to a Snackbar:

```
Snackbar.make(view, R.string.text_label, Snackbar.LENGTH_LONG)
    .setAction(R.string.action_text) {
        // Responds to click on the action
    }
    .show()
```



Floating Action Button (FAB)

- Perform the most-common action for a screen (for example, creating a new email)
- Come in multiple sizes (regular, mini, and extended)



CoordinatorLayout

- Acts as top-level container in an app
- Manages interaction of its child views, such as gestures
- Recommended for use with views like a Snackbar or FAB

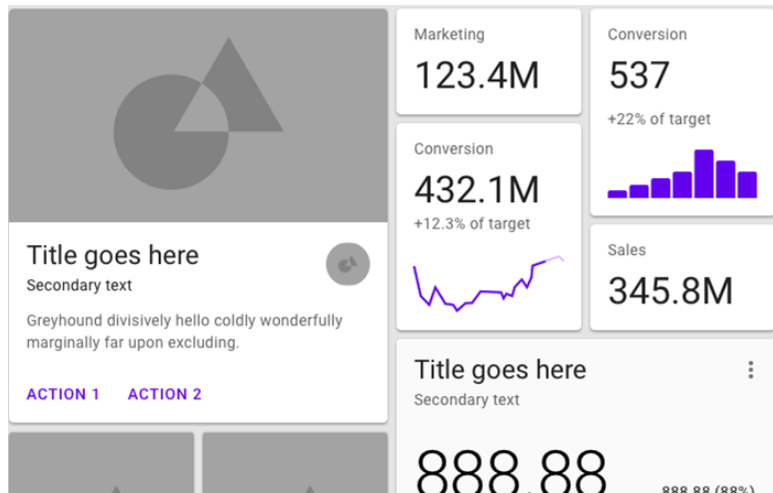
FAB example

```
<androidx.coordinatorlayout.widget.CoordinatorLayout ...>
    ....
    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/floating_action_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:layout_margin="16dp"
        android:contentDescription="@string/fab_content_desc"
        app:fabSize="normal" <!-- or mini or auto -->
        app:srcCompat="@drawable/ic_plus"/>

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Cards

- A card holds content and actions for a single item.
- Cards are often arranged in a list, grid, or dashboard.
- Use `MaterialCardView`.



MaterialCardView example

```
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <ImageView .../>
        <TextView .../>
    </LinearLayout>

</com.google.android.material.card.MaterialCardView>
```

Localization

Localize your app

- Separate the localized aspects of your app (for example, text, audio files, currency, and numbers) as much as possible from the core Kotlin functionality of the app.

Example: Extract the user facing strings into `strings.xml`.

- When a user runs your app, the Android system selects which resources to load based on the device's locale.
- If locale-specific resources are not found, Android falls back to default resources you defined.

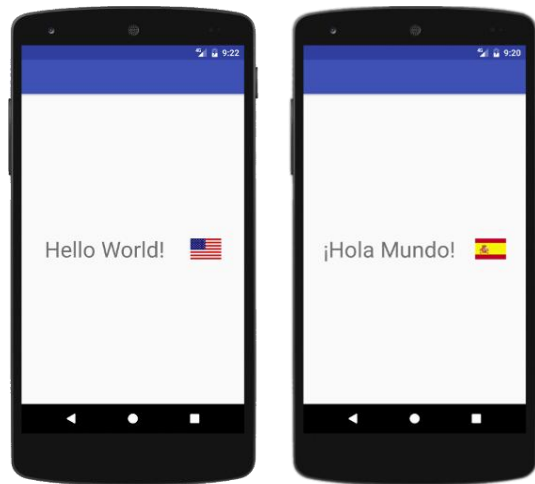
Support different languages and cultures

- Decide which locales to support.
- Create locale-specific directories in `res` directory:

```
<resource type>-b+<language code>  
    [+<country code>]
```

Examples: `layout-b+en+US`
`values-b+es`

- Provide locale-specific resources (such as strings and drawables) in those directories.



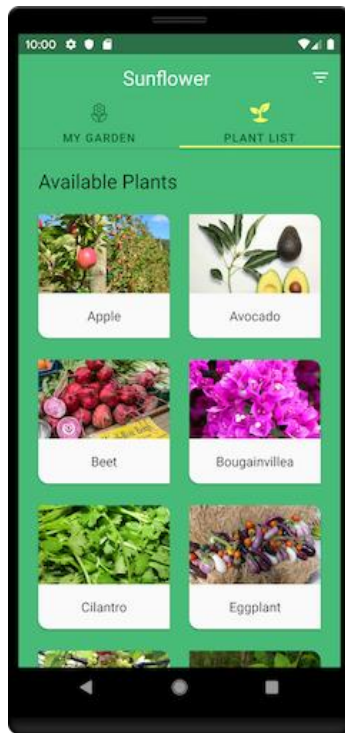
Support languages that use RTL scripts

- Users can choose a language that uses right-to-left (RTL) scripts.
- Add `android:supportsRtl="true"` to app tag in manifest.
- Convert `left` and `right` to `start` and `end`, respectively, in your layout files (change `android:paddingLeft` to `android:paddingStart`).
- Localize strings and format text in messages.
- Optionally, use `-ldrtl` resource qualifier to provide alternate resources.

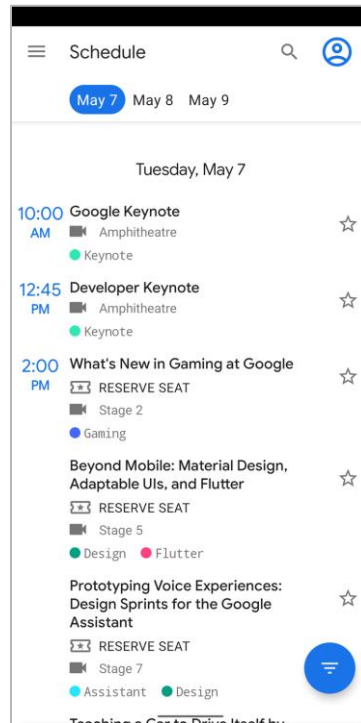
Example apps

Check out other apps

Sunflower
app



Google I/O
app



Demo

Assignment 1

Apply Shared Preference

2:03 PM 0.3KB/s

Day2Lab3Intent

Phone

Phone Number

Message

Your Message

CLOSE NEXT

2:03 PM 7.5KB/s

Day2Lab3Intent

Phone

01063208399

Message

any thing

CLOSE NEXT

10:45 PM 45.8KB/s

Day2Lab3Intent

Phone

01063208399

Message

any thing

Read SPref Write SPref

Close

1 2 ABC 3 DEF -

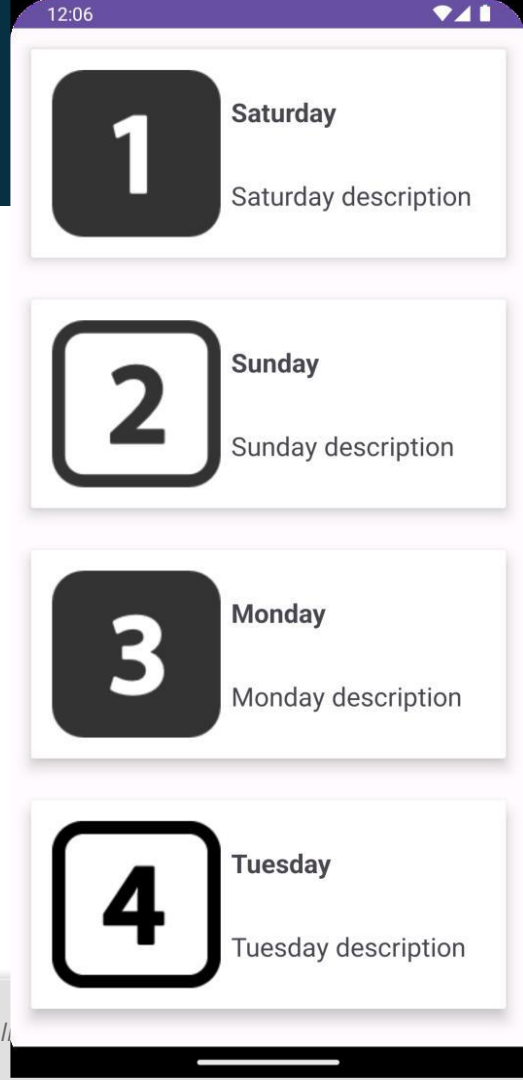
4 GHI 5 JKL 6 MNO ↵

7 PQRS 8 TUV 9 WXYZ ✕

*# 0 + . ➔

Assignment 2

Add card to the RecyclerView



Assignment 3 (Bonus)

Apply Data Store

2:03 PM 0.3KB/s

Day2Lab3Intent

Phone

Phone Number

Message

Your Message

CLOSE NEXT

2:03 PM 7.5KB/s

Day2Lab3Intent

Phone

01063208399

Message

any thing

CLOSE NEXT

10:45 PM 45.8KB/s

Day2Lab3Intent

Phone

01063208399

Message

any thing

Read SPref Write SPref

1 2 ABC 3 DEF -

4 GHI 5 JKL 6 MNO ↵

7 PQRS 8 TUV 9 WXYZ ✕

*# 0 + . ➔

Close